DSC550Week8.2

July 27, 2025

[162]:
```
# importing required packages and libraries for this excersize
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, GridSearchCV
import numpy as np
```

[164]:
```
# Load the data
try:
    data = pd.read_csv('./Loan_Train.csv')
except FileNotFoundError:
    print("Error: Loan_Train.csv not found. Make sure the file is in the same␣
 ↪directory or provide the correct path.")
    exit()
```

[166]:
```
# Display the Loan Train dataset

df = pd.DataFrame(data)
print(df)
```

```
      Loan_ID  Gender Married Dependents     Education Self_Employed  \
0    LP001002    Male      No          0      Graduate            No
1    LP001003    Male     Yes          1      Graduate            No
2    LP001005    Male     Yes          0      Graduate           Yes
3    LP001006    Male     Yes          0  Not Graduate            No
4    LP001008    Male      No          0      Graduate            No
..        …     …     …         …            …           …
609  LP002978  Female      No          0      Graduate            No
610  LP002979    Male     Yes         3+      Graduate            No
611  LP002983    Male     Yes          1      Graduate            No
612  LP002984    Male     Yes          2      Graduate            No
613  LP002990  Female      No          0      Graduate           Yes

     ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
0               5849                0.0         NaN             360.0
```

```
1               4583             1508.0       128.0             360.0
2               3000                0.0        66.0             360.0
3               2583             2358.0       120.0             360.0
4               6000                0.0       141.0             360.0
..               …                 …          …                 …
609             2900                0.0        71.0             360.0
610             4106                0.0        40.0             180.0
611             8072              240.0       253.0             360.0
612             7583                0.0       187.0             360.0
613             4583                0.0       133.0             360.0

     Credit_History Property_Area Loan_Status
0               1.0         Urban           Y
1               1.0         Rural           N
2               1.0         Urban           Y
3               1.0         Urban           Y
4               1.0         Urban           Y
..              …           …         …
609             1.0         Rural           Y
610             1.0         Rural           Y
611             1.0         Urban           Y
612             1.0         Urban           Y
613             0.0     Semiurban           N

[614 rows x 13 columns]
```

[168]:
```
# Step 1. Drop the column "Loan_ID"
df.drop(columns=['Loan_ID'], inplace=True)
```

[170]:
```
# Step 2. Drop rows with missing data
df.dropna(inplace=True)
```

[172]:
```
# Step 3: Convert categorical features into dummy variables

categorical_cols = df.select_dtypes(include='object').columns
df = pd.get_dummies(df, columns=categorical_cols, drop_first=True) # drop_first␣
 ↪avoids multicollinearity

display(df)
```

```
     ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
1               4583             1508.0       128.0             360.0
2               3000                0.0        66.0             360.0
3               2583             2358.0       120.0             360.0
4               6000                0.0       141.0             360.0
5               5417             4196.0       267.0             360.0
..               …                 …          …                 …
609             2900                0.0        71.0             360.0
```

```
610             4106                0.0        40.0             180.0
611             8072              240.0       253.0             360.0
612             7583                0.0       187.0             360.0
613             4583                0.0       133.0             360.0

     Credit_History  Gender_Male  Married_Yes  Dependents_1  Dependents_2  \
1               1.0         True         True          True         False
2               1.0         True         True         False         False
3               1.0         True         True         False         False
4               1.0         True        False         False         False
5               1.0         True         True         False          True
..              …           …            …            …             …
609             1.0        False        False         False         False
610             1.0         True         True         False         False
611             1.0         True         True          True         False
612             1.0         True         True         False          True
613             0.0        False        False         False         False

     Dependents_3+  Education_Not Graduate  Self_Employed_Yes  \
1            False                   False              False
2            False                   False               True
3            False                    True              False
4            False                   False              False
5            False                   False               True
..             …                     …                  …
609          False                   False              False
610           True                   False              False
611          False                   False              False
612          False                   False              False
613          False                   False               True

     Property_Area_Semiurban  Property_Area_Urban  Loan_Status_Y
1                      False                False          False
2                      False                 True           True
3                      False                 True           True
4                      False                 True           True
5                      False                 True           True
..                       …                   …              …
609                    False                False           True
610                    False                False           True
611                    False                 True           True
612                    False                 True           True
613                     True                False          False

[480 rows x 15 columns]
```

[174]:
```
# Step 4: Split the data into training and test sets

X = df.drop('Loan_Status_Y', axis=1)
y = df['Loan_Status_Y']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
↪random_state=42)
```

[176]:
```
# Step 5: Create a pipeline with Min-Max Scaler and KNN classifier
pipeline = Pipeline([
        ('scaler', MinMaxScaler()),  # Step 1: Scale features using MinMaxScaler
        ('knn', KNeighborsClassifier()) # Step 2: Apply KNN classifier
    ])
```

[178]:
```
# Step 6: Fit the pipeline to the training data:
pipeline.fit(X_train, y_train)
```

[178]:
```
Pipeline(steps=[('scaler', MinMaxScaler()), ('knn', KNeighborsClassifier())])
```

[180]:
```
# Step 7: Make predictions and evaluate the model on the test set:
y_pred = pipeline.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy on Test Set: {accuracy:.4f}")
```

```
Model Accuracy on Test Set: 0.7812
```

[148]:
```
pipeline_knn_default = Pipeline([('scaler', MinMaxScaler()), ('knn',␣
↪KNeighborsClassifier())])
pipeline_knn_default.fit(X_train, y_train)
accuracy_default_knn = pipeline_knn_default.score(X_test, y_test)
```

[150]:
```
# Fit a grid search with 5-fold cross-validation
param_grid_knn = {'knn__n_neighbors': range(1, 11)}
grid_search_knn = GridSearchCV(pipeline_knn_default, param_grid_knn, cv=5)
grid_search_knn.fit(X_train, y_train)
best_knn_model = grid_search_knn.best_estimator_
accuracy_best_knn = best_knn_model.score(X_test, y_test)
```

[154]:
```
# Expanded Grid Search
pipeline_multi_model = Pipeline([('scaler', MinMaxScaler()), ('classifier',␣
↪KNeighborsClassifier())]) # Placeholder
param_grid_multi = [
    {'classifier': [KNeighborsClassifier()], 'classifier__n_neighbors':␣
↪range(1, 11)},
    {'classifier': [LogisticRegression()], 'classifier__C': [0.1, 1, 10]}, #␣
↪Example
    {'classifier': [RandomForestClassifier()], 'classifier__n_estimators': [50,␣
↪100]} # Example
```

```
]
grid_search_multi = GridSearchCV(pipeline_multi_model, param_grid_multi, cv=5)
grid_search_multi.fit(X_train, y_train)
best_multi_model = grid_search_multi.best_estimator_
best_multi_params = grid_search_multi.best_params_
accuracy_best_multi = best_multi_model.score(X_test, y_test)
```

0.1 Summarize findings

[156]:
```
# Summarize results
print(f"Default KNN Accuracy: {accuracy_default_knn}")
print(f"Best KNN Accuracy: {accuracy_best_knn}, n_neighbors: {grid_search_knn.
  ↪best_params_['knn__n_neighbors']}")
print(f"Best Multi-Model Accuracy: {accuracy_best_multi}")
print(f"Best Multi-Model and Hyperparameters: {best_multi_params}")
```

```
Default KNN Accuracy: 0.78125
Best KNN Accuracy: 0.7916666666666666, n_neighbors: 3
Best Multi-Model Accuracy: 0.8229166666666666
Best Multi-Model and Hyperparameters: {'classifier': LogisticRegression(),
'classifier__C': 10}
```

[ ]:
```

```